

Week 7: Supply Chain Completa con Turn Engine y Agente Claude

1. Arquitectura del sistema

Esta semana añadimos la capa de retail y un orquestador central que mueve toda la simulación hacia adelante día a día de forma automática.

```

flowchart TD
    TE([Turn Engine])
    EC([Customer Demand Generator])
    RA([Retailer\nport 8003])
    MA([Manufacturer\nport 8002])
    PA([Provider\nport 8001])

    TE --> EC
    EC -->|POST /api/orders| RA
    TE -->|run agent/stub| RA
    TE -->|run agent/stub| MA
    TE -->|run agent/stub| PA
    RA -->|POST /api/orders| MA
    MA -->|POST /api/orders| PA
    TE -->|POST /api/day/advance| PA
    TE -->|POST /api/day/advance| MA
    TE -->|POST /api/day/advance| RA
  
```

Flujo de datos por turno: 1. Turn Engine lee las señales de mercado del scenario file 2. Genera N órdenes de clientes y las inyecta al Retailer 3. El agente/stub del Retailer decide cumplir stock y comprar al Manufacturer 4. El agente/stub del Manufacturer libera producción y compra materiales al Provider 5. El Provider stub no hace nada activo (el advance recibe entregas) 6. Turn Engine avanza el día en los tres apps en orden: Provider → Manufacturer → Retailer

2. Turn Engine: diseño y orden de roles

El orden downstream → upstream es clave: el Retailer actúa primero porque su demanda define qué necesita el Manufacturer, y el Manufacturer actúa antes que el Provider porque sus pedidos de materiales definen qué tiene que entregar el Provider.

Si invirtiéramos el orden (Provider → Manufacturer → Retailer), el Manufacturer estaría comprando materiales sin saber todavía cuántos pedidos entrarán del Retailer en ese turno, y el Provider avanzaría sin que el Manufacturer haya pedido nada aún. La simulación funcionaría pero sería menos reactiva.

El turn engine tiene dos modos: - **Stub**: decisiones deterministas vía HTTP directo (siempre libera todo lo que puede, compra si stock < 2) - **Agent**: invoca `claude --print` con el skill file y el contexto del día, captura el output en `logs/`

3. El Skill File

El archivo `skills/manufacturing-manager.md` enseña a Claude a comportarse como el gerente de fábrica. Contenido completo:

```
# Skill: Manufacturer Manager

## Your Role
Cada día recibes órdenes de retailers y decides qué producir y qué materiales comprar.

## Available Commands
- manufacturer-cli.py day current / stock / capacity
- manufacturer-cli.py sales orders [--status pending]
- manufacturer-cli.py production release <order_id>
- manufacturer-cli.py purchase create --supplier ... --product ... --qty ...
- manufacturer-cli.py price list / price set <model> <price>

## DO NOT
- DO NOT call day advance (el turn engine lo hace)
- No liberes más órdenes que la capacidad disponible

## Decision Framework
1. Assess: stock, capacidad, órdenes pendientes
2. Libera lo que puedes (BOM ok + capacidad ok)
3. Compra lo que falta (stock < 10 unidades sin PO abierto)
4. Ajusta precios si órdenes llevan 2+ días esperando
5. Resume en 3-5 bullets lo que hiciste y por qué

## When Done
Print resumen. No avances el día.
```

Dos decisiones de diseño al escribir este skill:

1. **La sección DO NOT es lo más importante.** Sin ella, Claude intentará "ayudar" y llamará a `day advance` por su cuenta porque es el paso lógico siguiente. Lo descubrí leyendo el PDF del reto que advertía

exactamente esto. Un LLM eager-to-please es peligroso en simulaciones con estado.

2. El BOM inline en el framework. En la primera versión del skill no ponía la receta de materiales y el agente inventaba necesidades de stock incorrectas (pedía pantallas LCD para el P3D-Classic que no las lleva). Añadir la BOM explícita en el Decision Framework hizo las decisiones de compra mucho más precisas.

4. Proof of Concept: Output del agente

Corrí el turn engine con `--agent manufacturer` para el día 1. El retailer inyectó 3 órdenes de clientes y compró 3x P3D-Classic y 3x P3D-Pro al manufacturer. El agente recibió 2 sales orders (id:1 y id:2) y produjo este output:

```
Turn Engine – smoke-test | Day 1 | normal (modifier=1.0)

[Demand] Injected: 3/3 customer orders accepted

[Retailer Stub] Ordered 3x P3D-Classic from manufacturer
[Retailer Stub] Ordered 3x P3D-Pro from manufacturer

[Agent] Running claude for manufacturer (day 1)...

Ordering 20 pcb from ChipSupply Co: stock will be 2 after production,
expected demand requires 3 for Pro order + buffer. Arrives day 4.

**Day 1 Summary:**
- 1 order released – Order #1 (P3D-Classic x3, $1,500): materials ok, capacity ok.
- 1 order held – Order #2 (P3D-Pro x3, $2,400): only 5 PCBs, need 6 total;
  Classic gets priority (arrived first, fewer parts).
- 1 purchase order placed – 20 PCBs from ChipSupply Co at $28.50/unit (tier price).
- Concern: PCB is the critical bottleneck – stock will drop to 2 after Classic production.
- Tomorrow: Classic x3 enters production, Pro order remains pending until day 4.
```

Qué hizo bien: - Entendió la restricción de materiales correctamente (PCBs son el cuello de botella) - Calculó bien la prioridad (Classic primero porque usa menos PCBs) - Explicó su razonamiento antes de cada mutación (audit trail limpio) - Calculó el precio por tier de forma correcta (\$28.50 es el tier de 20+ unidades)

Dónde fue inestable: - El log está truncado (solo aparece la parte final antes del resumen) porque Claude emite texto antes de ejecutar herramientas y el stdout se captura mezclado - No verificó si el proveedor tenía el stock disponible antes de ordenar — asumió que sí - En días posteriores (sin estado previo) puede perder contexto sobre pedidos anteriores

5. Notas de vibe-coding

Claude Code construyendo el software:

Lo que fue bien: crear las tres apps con estructura consistente desde la de week-06 fue muy rápido. Reusar el patrón SQLAlchemy + FastAPI + CLI de argparse del manufacturer para la retailer tardó minutos. Las partes de plomería (session management, seed loading, event logging) se copiaron y adaptaron sin errores.

Lo que fue mal: el path del seed file lo puse en `parents[2]` en lugar de `parents[1]` y el retailer arrancaba vacío sin dar error (solo devolvía catalogo vacío silenciosamente). Tardé un par de minutos en debuggear por qué las órdenes de cliente daban 422. Los tests me lo hubieran pillado antes si hubieran cargado el seed desde disco en lugar de en-memoria.

El agente ejecutando el rol:

Lo que fue bien: el agente entendió rápido la lógica de BOM y tomó decisiones razonables. El resumen final en bullets es muy limpio y útil para los logs. Siguió la restricción DO NOT (no intentó avanzar el día).

Lo que fue mal: el output mezclado con el razonamiento intermedio hace difícil parsear qué comandos ejecutó realmente. Para week-08 habría que capturar los `tool_use` events por separado del texto libre. También el agente tardó ~30 segundos en responder, lo cual es lento para simular múltiples días seguidos.